



Universidad de Jaén

Tema 4.2 Programación en el Cliente: Interacción con el Navegador

Desarrollo de Aplicaciones Web

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una [Licencia Creative Commons](https://creativecommons.org/licenses/by-sa/4.0/)
Atribución-CompartirIgual 4.0 Internacional.

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

v 2.5



Objetivos

- Conocer y saber utilizar el **modelo de ejecución** de javascript en el navegador
- Conocer y saber cómo acceder desde javascript a los elementos básicos del **API del navegador (BOM)**
- Conocer la estructura del **modelo de objetos del documento (DOM)** y saber manipularla usando Javascript y jQuery
- Saber gestionar correctamente **eventos** del navegador y del documento



Uso de Javascript en el Cliente

- En el documento cuerpo `<head>` o `<body>` del documento html:
 - `<script> código javascript </script>`
 - Ejecución síncrona del código mientras se carga la página
 - e.g. para añadir contenido "dinámico" a la misma **(no recomendado)** "unobstructive javascript"
- En URLs o atributos
 - `<a href="javascript:alert('No disponible');">`
 - `<button onclick="validarDatos()">Verifica</button>`
- Desde fichero externo: `<script src="fichero.js"></script>`
 - Definición de funciones, objetos y variables globales
 - Es "cacheado" por el cliente. Velocidad de carga
 - Descarga desde cualquier servidor
 - **Carga y ejecución**
 - *Por defecto:* cuando se encuentra la etiqueta (tradicionalmente colocado al final de *body* para acelerar la visualización de la página al cliente) **(obsoleto, no recomendado)**
 - Es recomendable ubicarlo en etiqueta `<head>` controlando la carga y ejecución con atributos:
 - atributo **async**, carga simultánea a html y ejecución asíncrona
 - atributo **defer**, carga simultánea, ejecución (síncrona) al finalizar la carga y procesamiento de html `<script defer src="fichero.js"></script>`
 - ES6 permite la inclusión de **módulos en ficheros JS**

3

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Módulos ES6

- **Solventan algunas carencias de scripts JS**
 - Definición de funciones, objetos y variables locales al módulo
 - **Sintaxis específica:** modo estricto, declaración de variables obligatoria,
 - Rutas de descarga relativa (local) o desde servidores con CORS habilitado
 - Carga y ejecución tipo **defer**: ejecuta al finalizar carga y procesamiento de html
 - Pueden a su vez **cargar otros módulos**
 - Permiten **importar** elementos de otros módulos y **exportar** elementos propios a otros
- Uso: Soportados en navegadores actuales con `<script type="module">`

Ejemplo:

```
// js/utilidades.js
const dtoBase = 0.5;           //local al módulo
const dtoActual = dtoBase + 0.3;
export dtoActual;               //visible fuera del módulo
export function calculaPVP ( precio ) {
    return precio*(1+dtoActual);
}

// js/app.js
import { dtoActual, calculaPVP } from ".utilidades.js";

console.log( `Descuento actual ${dtoActual}` );
```

3

2

```
// index.html
<html>
<head>
  <script type="module" src=".js/app.js" />
</head>
<body>
  ...
</body>
</html>
```

1

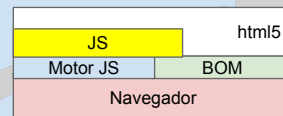
4

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Browser Object Model, BOM

- Es el conjunto de APIs en el navegador al que Javascript tiene acceso mediante objetos globales predefinidos o funciones constructoras



● API del Navegador

- **window** es el prototipo del objeto global en el navegador
 - Contiene las variables "globales" de javascript y funciones API navegador
 - También accesible mediante la propiedad: window


```
alert("Aviso")
window.alert("aviso")
```
- Cada ventana tiene su propia copia del objeto Window
 - Las variables "globales" y métodos definidos permanecen en su ventana
- Algunos métodos
 - **alert(msg)** //ventana con mensaje de aviso
 - **prompt(msg, valorDefecto)** //devuelve valor introducido
 - **confirm(msg)** //devuelve verdadero o falso

5

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Acceso al navegador: propiedades

● Algunas propiedades de **window**

- **innerHeight, innerWidth** //Dimensiones contenido ventana
- **outerHeight, outerWidth** //Dimensiones ventana (tool&scroll)
- **pageXOffset, pageYOffset** //Desplazamientos en px (scroll)
- **screenLeft, screenTop** //Posición ventana navegador
- **screenX, screenY** // pos=window.screenX|window.screenLeft
- **status** //Texto barra de estado
- **screen** //objeto con detalles sobre dispositivo
 - **width, height, colorDepth**
- **document** //Contenido del documento
 - **lastModified** //fecha de modificación documento en servidor
 - **referrer** //URL del documento que apunta al actual
 - **title** //Título del documento
 - **url** //URL del doc actual
 - **documentElement** //Raíz del árbol de componentes de la página! (nodo <html>)
- **console** //Acceso a ventana de depuración del navegador
 - **log(msg o variable)** //Muestra un mensaje u elemento en consola

6

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Acceso al navegador

- El origen del documento se controla con el objeto **window.location**
 - **location.href** //leer la ubicación o cargar una nueva
 - href.protocol, host, hostname, port, pathname, search

Ejemplo:

```
<html>
<head>
  <script>
    location.href="servicio_no_disponible.html";
  </script>
...
```

- **window.history**, control del histórico de la ventana
 - No se puede leer el contenido, sólo interactuar
 - history.back(), forward(), go(+/-num)
- **window.navigator**, devuelve información sobre el navegador.

7

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Algunos objetos APIs HTML5

HTML5 añade elementos adicionales al API para mejorar el acceso a los recursos del sistema operativo

- **window.localStorage**
 - Acceso a almacenamiento ampliado en el navegador
- **window.indexedDB**
 - Creación y acceso a bases de datos simples en el navegador
- **window.Worker**
 - Control de hebras de ejecución en Javascript
- **window.applicationCache**
 - Control de caché de contenidos de la app web
- **window.navigator.geolocation**
 - Acceso a localización del dispositivo
- **window.navigator.getBattery**
 - Control estado de la batería

El objeto sólo existirá si el API está disponible

- 8 [Modernizer Cross-Browser HTML5 polyfills](#) Bibliotecas JS con emulaciones APIs HTML5

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



<https://jquery.com>



- Presentada en Enero de 2006 por John Resig
- Actualmente biblioteca JS más utilizada en internet, con un [77% de uso](#)
- Diseñada para simplificar la programación en el cliente cuando las APIs de los navegadores tenían diferencias importantes
- **Características:**
 - Compatibilidad entre navegadores (1.x IE6+, 3.x IE9+)
 - Selección simplificada de elementos del DOM
 - Manipulación del DOM
 - Gestión homogénea de eventos
 - Soporte Ajax
 - Efectos y animaciones
 - Utilidades de apoyo a Javascript
 - Extensibilidad mediante Plug-ins
- Gracias a la mayor compatibilidad entre navegadores actuales, **en muchas ocasiones ya no es necesaria** <http://youmightnotneedjquery.com/>

9

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Uso de jQuery

`<script src="js/jquery.js"></script>`



- **Versión minimizada** para entornos de producción:
 - *jquery.min.js*, sin comentarios, identificadores simplificados a,b,c... espacios eliminados al máximo
 - versión 1.12 : 293 KB normal vs 92 KB minimizada
 - versión 3 (IE9+):
 - normal, ~270KB vs ~87KB minimizada
 - slim (sin AJAX ni animaciones)
 - En desarrollo es preferible la versión normal para depuración
- Se puede utilizar el código de versión en el fichero, pero si se actualiza deben modificarse muchas páginas...
 - `<script src="js/jquery-3.7.1.js"></script>`
- Pueden utilizarse **CDN** (Content Delivery Networks) para mejorar la velocidad de carga... ¿seguridad?
`<script src="//code.jquery.com/jquery-3.7.1.min.js" integrity="sha256..."></script>`
`<script src="//code.jquery.com/jquery-3.7.1.slim.min.js"></script>`

10

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Document Object Model DOM

Modelo de Objetos de Documento

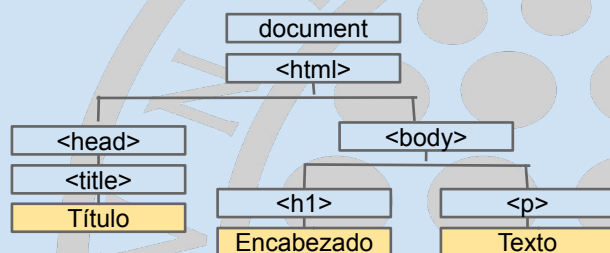
Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Modelo de Objetos de Documento, DOM

- Es la estructura de datos que representa el documento en la ventana del navegador
- Accesible a través del objeto `window.document`
- El modelo se utiliza también para procesar documentos XML
- Básicamente es un **Árbol** que representa el anidamiento de las etiquetas html (elementos)

```
<html>
  <head><title>Título</title></head>
<body>
  <h1>Encabezado</h1>
  <p>Texto</p>
</body>
```



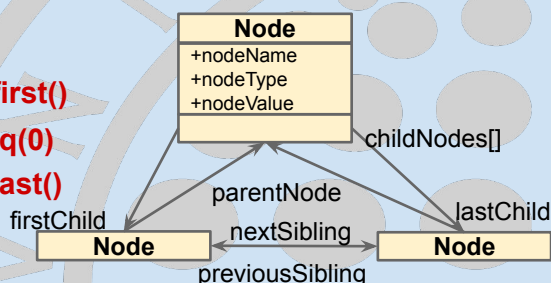
Estructura Árbol DOM

• Nodos del documento

- Los nodos del DOM disponen de atributos y métodos para su manipulación
- Referencias a su Padre, Nodos hijos y a sus hermanos
- jQuery** necesita encapsular los nodos del DOM para poder utilizar las propiedades y métodos de jQuery (*decorator pattern*)
 - jQuery(nodo)** o **\$(nodo)**
 - let \$nodo=\$('selector')** referencia a nodo jquery (convención de nombre)

• Atributos con relaciones entre nodos del DOM

- parentNode** **\$nodo.parent()**
- childNodes[]** **\$nodo.children()**
- firstChild** **\$nodo.children().first()**
- lastChild** **\$nodo.children().last()**
- nextSibling** **\$nodo.next()**
- previousSibling** **\$nodo.prev()**



13

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Accediendo a elementos del DOM

• Selección de nodos

- Se utilizan métodos de *window.document*
- getElementById("id")**
 - Referencia al nodo con atributo id indicado
- getElementsByName("name")**
 - Array de nodos con atributo name especificado. e.g. radiobuttons
- getElementsByTagName("tagname")**
 - Array de etiquetas del tipo tagname. * para todas!
- getElementsByClassName("style_class")** (IE>8)
- querySelector** y **querySelectorAll** (IE>7) **jQuery: \$('selector')**
 - se pueden utilizar selectores css!
 - ¿Nombre de métodos demasiado largos?

```
let el = selector => document.querySelector(selector);
el('#msg').textContent = "You might not need jquery"; $('#msg').text("You need jquery");
```



• Ejemplos

```
let d=document;
let mensajeDiv =d.getElementById ( "mensaje" );
let preferencias =d.getElementsByName ( "pref" );
let parrafos =d.getElementsByTagName ( "p" );
let elementos =d.getElementsByClassName ( 'warning' )
let items =d.querySelectorAll ( '#listClientes li' );
```

```
//varios checkboxes
//Todos los párrafos
//por clase
//Por selector
```

```
$('#mensaje')
$('[name="pref"]')
$('p')
$('.warning')
$('#listClientes li')
```

14

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Acceso a atributos en nodos

• Propiedades de Nodos de tipo Elemento

- Disponen de **propiedades correspondientes a atributos** de cada etiqueta html. e.g. *id, src, style, name*, etc.
- Algunas tienen **nombres adaptados**:
 - para evitar palabras reservadas en JS. e.g. *label.htmlFor* o *p.className*
 - con nombres compuestos: *defaultChecked, tabIndex*

`$( 'selec' ).prop( 'prop.' )` `$( 'selec' ).prop( 'prop.', valor )`

• Métodos para acceso a atributos de etiquetas HTML:

- `getAttribute("atributo")` `$( 'selec' ).attr( 'atrib' )`
- `setAttribute("atributo", valor)` `$( 'selec' ).attr( 'atrib', valor )`
- `.style.propiedad=valor` `$( 'selec' ).css( 'propiedad' [, valor ] )`
- `.classList.add("clase")` `$( 'selec' ).addClass( "clase" )`

• Ejemplos

```
var usuario=document.getElementById("nombre").value;
var usuario=$( '[name="nombre"]' ).prop( 'value' );
var usuario=$( '[name="nombre"]' ).val();
document.querySelector( '[name="nombre"]' ).className='error';
$( 'article' ).css( 'color', 'red' );
$( 'table#resultados tr' ).addClass( 'resaltado' );
```

Se admiten **atributos personalizados**, e.g.

```
<li id="prod1" data-precio="990">Iphone 15</li>
el( "#prod1" ).getAttribute( 'data-precio' )
el( "#prod1" ).dataset.precio=1000
$( "#prod1" ).data( 'precio' )
```

15

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Modificando contenido de Nodos

• Contenido+html

- nodo.**innerHTML** `$( 'selec' ).html( 'html content' )` `$( 'selec' ).empty()`
 - Contenido dentro de una etiqueta
- nodo.**outerHTML**
 - Etiqueta html y su contenido

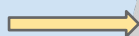
• Contenido en texto plano (sin html)

- **textContent, innerText**
 - `var cadena=elem.textContent;`
 - `$( 'selec' ).text()` o `$( 'selec' ).text( 'nuevo' )`

Ejemplos

```
document.getElementById("msgCliente").innerHTML="<p>No válido</p>";
$( '#msgCliente' ).html( '<p>No válido</p>' );
```

```
<body>
  <div id="msgCliente">
  </div>
</body>
```



```
<body>
  <div id="msgCliente">
    <p>No válido</p>
  </div>
</body>
```

16

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Modificando el DOM

• Métodos para crear un nodo

- `document.createElement("etiqueta")` `$('<etiqueta>')`
- `node.cloneNode(bool)` `$('sel').clone(bool)`
 - Crea una copia de un nodo. *true*: con descendientes

• Insertar Nodos

- `parentNode.appendChild(newnodo)` `$('sel').append(...)`
 - Coloca el nodo como su último hijo (más a la derecha)
- `parentNode.insertBefore(newnodo,hermano)` `$('sel').before()`
 - Coloca nuevo nodo antes de un hermano en la lista de hijos (childNodes)

• Eliminar nodos

- `parentNode.removeChild(nodoHijo)` `$('slchild').detach()`
 - devuelve el nodo extraído.
- `parentNode.replaceChild(newhijo,oldhijo)` `$('slchild').replaceWith(..)` `$('slchild').remove()` elimina eventos!

17

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo

//Añadir un nuevo párrafo con texto a un contenedor div

```
function nuevoParrafo(texto) {
    let nuevoP = document.createElement("p");
    nuevoP.textContent = texto;
    document.getElementById("msgs").appendChild(nuevoP);
}
```

//jQuery version

```
function nuevoParrafo(texto) {
    let $parrafo=$('<p/>').text(texto);
    $('#msgs').append($parrafo);
}
```

```
<html>
<body>
  <div id="msgs">
  </div>
</body>
</html>
```

↓

```
<html>
<body>
  <div id="msgs">
    <p>texto</p>
  </div>
</body>
</html>
```

18

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo

Añadir filas a una tabla de productos:

`productos=[ {nombre:"tuercas",precio:6 } , {nombre:"clavos",precio: 9}, ...];`

```
function rellenaTablaProductos(productos) {
  let tabla=document.createElement("table");
  productos.forEach( prod => {
    let fila=tabla.insertRow();
    for(let propiedad in prod ) {
      fila.insertCell().textContent=prod[propiedad];
    }
  });
  document.getElementById("productos")
    .appendChild(tabla);
}
```

```
<html>
  <body>
    <div id="productos"></div>
  </body>
</html>
```

//jQuery version

```
function rellenaTablaProductos(productos) {
  let $tabla= $('<table/>');
  productos.forEach( prod => {
    let $fila= $('<tr/>');
    $tabla.append( $fila );
    for(let propiedad in prod) {
      $fila.append( $('<td/>').text( prod[propiedad] ) );
    }
  });
  $('#productos').append( $tabla );
}
```

tuercas	6
clavos	9

Modelo de ejecución en el cliente

Modelo de ejecución en cliente

- **Ejecución orientada a eventos:** cuando se ha cargado el documento y/o recursos asociados (javascript, css, ...)
- **Manejadores de eventos:** durante la carga del documento se asocia código javascript a la ocurrencia de ciertos sucesos en el navegador, documento o recursos asociados
 - e.g. pulsación de una tecla, movimiento del ratón, etc.
- Un manejador recibe un **objeto event** con información sobre el evento que debe atender
- **Los eventos se propagan (bubbling)** del elemento que los recibe hacia los elementos que lo contienen (rama en DOM)
- Algunos eventos tienen un **comportamiento por defecto** que el manejador puede sustituir o complementar
- El modelo de eventos de jQuery homogeniza(ba) **incompatibilidades entre navegadores**

21

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Registro de manejadores de eventos

- Asignar código a una **propiedad de un objeto** o elemento del documento

(Las propiedades comienzan por **on**evento)

```
window.onload= () => {
    let frm=document.forms["fichaCliente"];
    frm.onsubmit = () => {
        return validar(); }
}
```

```
$(window).on("load", () => {
    $('form[name="fichaCliente"]')
        .submit( () => {
            return validar(); });
})
```

- **Registro estándar de manejadores como Listeners (IE>8)**

- Permite varios manejadores para un mismo evento sobre el mismo objeto
- **node.addEventListener("tipo_ev", evhandler [, captura])**
- **node.removeEventListener("tipo_ev", evhandler [, captura])**
 - **eventos** sin prefijo on
 - **captura=true**, atender eventos antes que elementos subordinados
- **\$(selector).on("tipo_ev tipo_ev2 ", evhandler);**
- **\$(sel').off("tipo_ev tipo_ev2", evhandler)**

```
myBtn.addEventListener( "click", function () {alert(..);} );
```

```
$(sel').on("click dblclick", function () {alert(..);} );
```

22

Capture Bubbling



Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Manejo de Eventos

- Cada función manejadora recibe como primer parámetro un **objeto Event**
 - jQuery dispone de un objeto Event propio para normalizar diferencias entre navegadores, `$event.originalEvent`
- **Propiedades de un evento**
 - **type**: cadena que identifica el tipo de evento. e.g. *click*, *mousemove*, *load*, ...
 - **target**: **elemento donde ocurre el suceso** o al que está asociado. e.g. *un botón*, *un enlace*, etc.
 - **atributos específicos** según evento.
 - **pageX**, **pageY** para eventos de ratón
 - **which**, botón o tecla pulsada

23

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Control ejecución de eventos

- Algunos eventos tienen un **comportamiento por defecto**. e.g. *"click" en un enlace o submit*
- **Control comportamiento por defecto:**
 - **Manejador en atributo, propiedad** de objeto, e.g. `onsubmit`
 - Si devuelve **true**, se activa comportamiento por defecto. e.g. enviar formulario
 - Si devuelve **false**, se cancela
 - **Manejador como listener**.
 - **`event.preventDefault()`**
 - Anula el comportamiento por defecto, e.g. submit de un formulario
 - **`event.stopPropagation()`**
 - Evita que se avise a elementos superiores salvo manejadores ya registrados

24

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Ejemplo de manejo de eventos (I)

- Versión formulario JSF de edición de cliente con validación en cliente con dos formas de asignar manejador de validación
- IMPORTANTE en JSF**
 - Se añade el id del formulario al de cada control, e.g. fCliente:idNombre
 - IMPORTANTE:** En jQuery o querySelector, el selector # debe escapar el carácter :, e.g. #fCliente\\:idNombre

```
/cliente/edita.xhtml
<h:outputScript library="js" name="validacliente.js"/>

<h:form id="fCliente"
  onsubmit="validarFormulario" >
  <h3>ID: #{clienteCtrl.cliente.id} /></h3>
  <h:inputHidden value="#{clienteCtrl.cliente.id}"/>

  <label>Nombre:</label>
  <h:inputText id="idNombre"
    value="#{clienteCtrl.cliente.nombre}" />
  <p id="errNombre"><h:message for="idNombre" /></p>
  <label>DNI:</label>
  <h:inputText id="idDNI"
    value="#{clienteCtrl.cliente.dni}"/>
  <p id="errDNI"><h:message for="idDNI" /></p>
  <label>Socio:</label>
  <h:selectBooleanCheckbox
    value="#{clienteCtrl.cliente.socio}" />
  <h:commandButton value="Guardar"
    action="#{clienteCtrl.guarda}"/>
</h:form>
```

opción 2

```
/resources/js/validacliente.js
let el = selector => document.querySelector(selector);
window.addEventListener('load', () => {
  el("#fCliente")
    .addEventListener('submit', validarFormulario);
});
function validarFormulario(event) {
  event.preventDefault(); //stop form submit
  let nombre=el("#fCliente\\:idNombre").value;
  let dni = el("#fCliente\\:idDNI").value;

  let valido = true;
  if (nombre.length < 3 || nombre.length > 50) {
    el("#errNombre")
      .textContent = "La longitud del nombre no es válida";
    valido = false;
  }
  if (dni.search(/^[0-9]{7,8}(-?[a-z])?$/i) === -1) {
    el("#errDNI")
      .textContent = "DNI no válido (12345678-A)";
    valido = false;
  }
  if (valido) el("#fCliente").submit(); //continue form submission
```

opción 1

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo de manejo de eventos (II)

Versión formulario JSP de edición de cliente con validación en cliente **con jQuery**

/WEB-INF/clientes/edita.jsp

```
<body>
<h1>Editar Cliente con validación en cliente</h1>
<form method="POST">
  Cliente nº: #{cliente.id}<br>
  <input name="id" type="hidden" value="#{cliente.id}">
  <label>Nombre: <input name="nombre"
    value="#{cliente.nombre}"></label>
  <span id="errNombre">#{errNombre}</span><br>
  <label>DNI:<input name="dni" value="#{cliente.dni}">
  </label><span id="errDni">#{errDni}</span><br>
  <label>Socio:<input name="socio" type="checkbox"
    value="#{cliente.socio?checked}:">
  </label><br>
  <input name="enviar" type="submit" value="Guardar">
  <input name="enviar" type="reset" value="Limpiar">
  <a href="listado">Volver</a>
</form>
<!--Load script at the end to show html content quickly-->
<script src="js/libs/jquery/jquery.js"></script>
<script src="js/edita.js"></script>
</body>
```

/js/edita.js

```
$(document).ready(function () {
  //Register component events
  $('form').submit(validarCliente);
});

function validarCliente (event) {
  var valido=true;
  var nombre=$('#name="nombre").val().trim();
  var dni=$('#name="dni").val().trim();
  if (nombre.length<3 || nombre.length>50) {
    $('#errNombre').text("La longitud del nombre
    no es válida");
    valido=false;
  }
  if (dni.search(/^[0-9]{7,8}(-?[a-z])?$/i)===-1) {
    $('#errDni').text("DNI no válido (12345678-A)");
    valido=false;
  }
  return valido;
}
```

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Algunos eventos del navegador

• window

- **.unload** `$(window).on('load', handler)`
 - Se activa cuando se ha terminado de cargar el documento y todos sus elementos relacionados
- **.onbeforeunload/onunload** `$(window).on('beforeunload', handler),
.on('unload', handler)`
 - antes (cancelable si *false*)/al salir de una página
- **id=setTimeout/setInterval**(function , milisecs)
- **clearTimeout/clearInterval**(id)
 - No son eventos, pero permiten ejecutar una función con un retraso o cada cierto tiempo

```
var idClockHnd= setInterval( function() {  
    $('#clock').text( (new Date()).toLocaleTimeString() );  
}, 1000);
```

• document

- **.DOMContentLoaded**, Sólo se ha cargado y analizado el documento (>IE8)
- **\$(document).ready(handler)** o **\$(handler)**

27

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Efectos visuales en jQuery

- Métodos que afectan propiedades CSS de nodos del DOM
- Todos los métodos pueden recibir la duración en realizar el cambio: en milisegundos o cadenas "slow", "fast"

- **show(), fadeIn(), slideDown()**
 - visualizar elementos ocultos
- **hide(), fadeOut(), slideUp()**
 - ocultar elementos (el espacio en pantalla se pierde)
- **toggle(), fadeToggle(), slideToggle()**
 - Cambiar estado de visualización
- **delay(mseg)**
 - retraso adicional entre varias opciones

Alternativas con API JS navegadores

http://youmightnotneedjquery.com/#fade_in

```
function msgTemporal(texto) {  
    $('#msg').text(texto).show('fast').delay(2000).fadeOut("slow");  
}
```

28

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Conclusiones

- Un navegador proporciona a Javascript un entorno de ejecución seguro (**Sandbox**) y un API, más o menos estándar, para el acceso al modelo de objetos del navegador (**BOM**) y, en particular, a la representación del documento HTML en memoria (**DOM**)
- Javascript en el navegador utiliza un modelo de **ejecución orientada a eventos**
- Tradicionalmente, bibliotecas como *jQuery* sirven para homogeneizar las características del API del navegador y simplificar operaciones de uso habitual
- La evolución homogénea del API JS de los navegadores actuales hace que bibliotecas como JQuery no sean tan necesarias como en años anteriores

Bibliografía

- Zakas, Nicholas C. Understanding ECMAScript 6 : the definitive guide for JavaScript developers. No Starch Press, 2016. Available on [[Recursos electrónicos UJA](#)]
- Rebecca Murphey. *Fundamentos de JQuery* (Leandro D'Onofrio). Uniwebsidad, 2006. Retrieved March 2025 from [[archive.org](#)]

Lecturas Complementarias

- *jQuery API Reference*. jQuery Foundation, 2019. Retrieved March 2024 from <http://api.jquery.com>
- *Javascript and HTML DOM reference*. W3schools. Retrieved March 2024 from <http://www.w3schools.com/jsref>